CUSTOMER BEHAVIOUR ANALYSIS

1. Jupyter Notebook – Python Data Preparation and EDA

The Jupyter Notebook is the first stage of the project. It loads the CSV dataset, checks data quality, handles missing review ratings, standardises column names, explores purchasing behaviour and prepares the DataFrame for PostgreSQL.

1.2 Import libraries and load data

```
import numpy as np
```

```
import pandas as pd
```

```
df=pd.read_csv('customer_shopping.csv')
```

1.3 Missing-value treatment

```
df['Review Rating']=df.groupby('Category')['Review Rating'].transform(
    lambda x : x.fillna(x.median())
)
```

1.4 Standardize column names

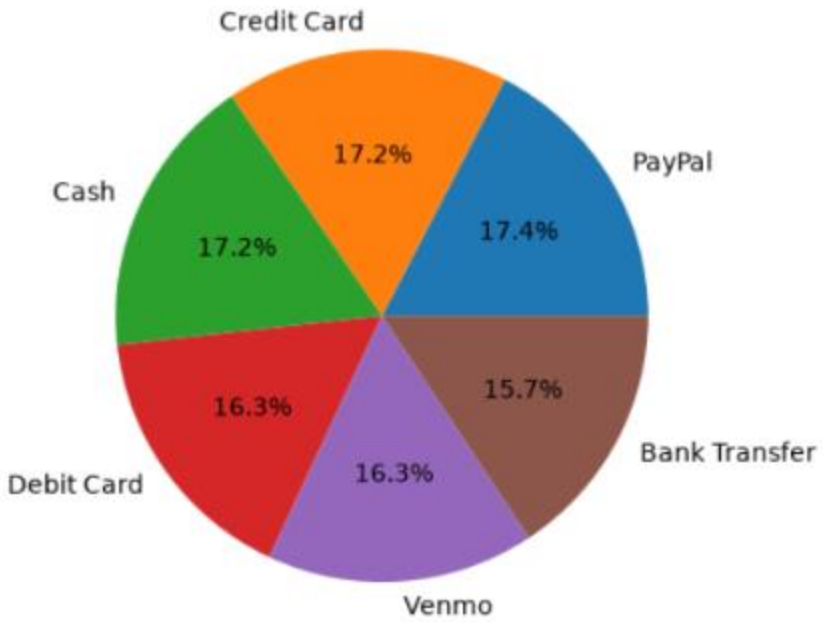
```
df.columns = (df.columns
              .str.strip()
              .str.lower()
              .str.replace(' ', '_'))
```

Column names are trimmed, converted to lowercase and spaces are replaced with underscores. The purchase amount field is shortened to purchase_amount for easier SQL querying.

1.5 Exploratory data analysis and visualisation

```
df['payment_method'].value_counts().plot(kind='pie',autopct='%1.1f%%')
```

<Axes: >



1.6 Load the prepared data into PostgreSQL

```
from sqlalchemy import create_engine



username = "postgres"
password = "7373"
host = "localhost"
port = "5432"
database = "customer"

engine = create_engine(
    f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}"
)



table_name = "customer1"

df.to_sql(
    table_name,
    engine,
    if_exists="replace",
    index=False
)

print(f"Data successfully loaded into table '{table_name}' in database '{database}'.")
```

```
Data successfully loaded into table 'customer1' in database 'customer'.
```

2. PostgreSQL – SQL Queries and Business Analysis

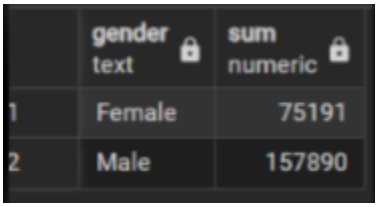
After loading the cleaned DataFrame into PostgreSQL, SQL is used to aggregate customer and transaction data. The following queries are based on the SQL supplied for this project,

2.1 View all records

Displays all rows and columns stored in the customer1 table.

2.2 Revenue by gender

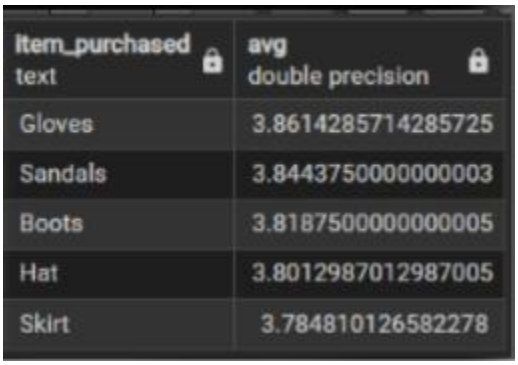
```
SELECT gender, SUM(purchase_amount)
FROM customer1
GROUP BY gender;
```



	gender text	sum numeric
1	Female	75191
2	Male	157890

2.3 Top five products by average review rating

```
SELECT item_purchased, AVG(review_rating)
FROM customer1
GROUP BY item_purchased
ORDER BY AVG(review_rating) DESC
LIMIT 5;
```



item_purchased text	avg double precision
Gloves	3.8614285714285725
Sandals	3.8443750000000003
Boots	3.8187500000000005
Hat	3.8012987012987005
Skirt	3.784810126582278

2.4 Average purchase amount by shipping type

```
SELECT shipping_type,
ROUND(AVG(purchase_amount), 2) AS avgg
FROM customer1
WHERE shipping_type IN ('Express', 'Standard')
GROUP BY shipping_type;
```

	shipping_type text	avgg numeric
1	Standard	58.46
2	Express	60.48

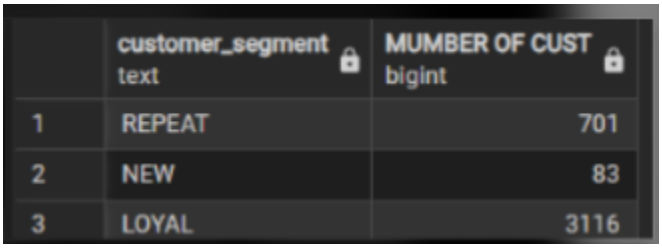
2.5 Subscriber versus non-subscriber performance

```
SELECT subscription_status,
AVG(purchase_amount),
SUM(purchase_amount),
COUNT(customer_id) AS total
FROM customer1
GROUP BY subscription_status;
```

subscription_status text	avg numeric	sum numeric	total bigint
No	59.8651211801896733	170436	2847
Yes	59.4919278252611586	62645	1053

2.6 Customer segmentation using previous purchases

```
SELECT subscription_status,
AVG(purchase_amount),
SUM(purchase_amount),
COUNT(customer_id) AS total
FROM customer1
GROUP BY subscription_status;
```



	customer_segment text	MUMBER OF CUST bigint
1	REPEAT	701
2	NEW	83
3	LOYAL	3116

2.7 Corrected SQL to calculate age-group revenue contribution

```
WITH age_group AS (
   SELECT purchase_amount,
      CASE
         WHEN age BETWEEN 20 AND 30 THEN 'GEN_Z'
         WHEN age BETWEEN 31 AND 50 THEN 'MILLENIALS'
         WHEN age BETWEEN 51 AND 100 THEN 'BOOMERS'
         ELSE 'OTHERS'
      END AS age_group
   FROM customer1
)
SELECT age_group,
      SUM(purchase_amount) AS total_revenue,
      ROUND(100.0 * SUM(purchase_amount) /
          SUM(SUM(purchase_amount)) OVER (), 2) AS revenue_pct
FROM age_group
GROUP BY age_group
ORDER BY total_revenue DESC
```

	age_groupp text	groupp bigint
1	OTHERS	150
2	MILLENIALS	1475
3	BOOMERS	1476
4	GEN_Z	799

3. Power BI – Dashboard and Report Template

The template includes KPI card visuals, column and bar charts, donut charts, a line-and-column combination visual, slicers, shapes, a text box and an image asset. The project dataset can be loaded and the visuals connected to the customer1 data model.

3.1 Data connection and preparation

Import the prepared customer_shopping.csv file or connect Power BI to the PostgreSQL customer database and select customer1.

Confirm data types: Customer ID as an identifier; Age and Previous Purchases as whole numbers; Purchase Amount as currency/decimal; Review Rating as decimal; and categorical columns as text.

Check that the Power BI model contains the cleaned column names used by the SQL table.

Create a one-row-per-customer dimension only if the data model requires it; otherwise use the transaction-level table consistently.

3.2 Suggested dashboard measures

```
Total Revenue = SUM(customer1[purchase_amount])

Total Records = COUNTROWS(customer1)

Average Purchase = AVERAGE(customer1[purchase_amount])

Average Rating = AVERAGE(customer1[review_rating])

Customer Count = DISTINCTCOUNT(customer1[customer_id])

Subscriber Count =

CALCULATE(

   DISTINCTCOUNT(customer1[customer_id]),

   customer1[subscription_status] = "Yes"

)
```

3.3Dashboard interpretation

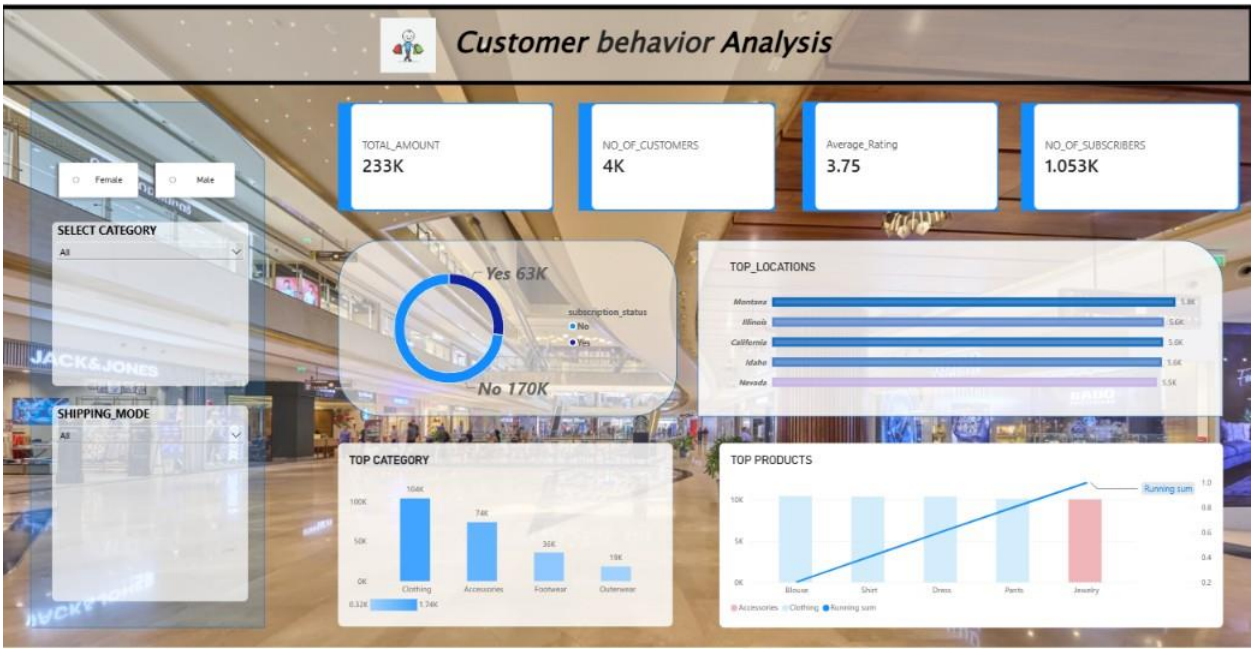
Male customers contribute $157,890 compared with $75,191 from female customers in the supplied SQL output.

Express shipping customers show an average purchase amount of $60.48 compared with $58.46 for Standard shipping.

Non-subscribers account for 2,847 records and $170,436 in purchase amount; subscribers account for 1,053 records and $62,645.

The supplied customer segmentation query classifies 3,116 records as LOYAL, 701 as REPEAT and 83 as NEW.

The top five products by average review rating are Gloves, Sandals, Boots, Hat and Skirt.



4. Conclusion

This project demonstrates an end-to-end customer shopping analysis workflow. Python is used to inspect and prepare the dataset, PostgreSQL is used to query and aggregate customer behaviour, and Power BI is used to communicate the results through an interactive dashboard. The SQL outputs highlight differences in revenue by gender, purchase amount by shipping type, subscription behaviour, product ratings, customer segments and age-group counts.

The main improvement before final submission is to ensure that the age-group revenue question uses SUM(purchase_amount), not COUNT(*), and to validate the purchase-frequency mapping. The Power BI report should use clearly defined measures and distinguish record counts from distinct customer counts.

THANK YOU